

Training Assignment — Ananya

Python Basics • Data Cleaning • Web Scraping (Practice site: books.toscrape.com)

Hi Ananya, this is your 1-week, intensive, self-paced learning-and-execution plan. You'll build three core skills — Python fundamentals, data cleaning with pandas, and web scraping — then apply them on a live practice project by scraping books.toscrape.com, a website built specifically for scraping practice (no legal/ethical concerns, unlike scraping a real commercial site).

This plan is being run in parallel with **Pavanaj**, who is following the same structure with a different set of practice exercises and a different scraping scope, so your two outputs can be merged at the end without any duplicate work.

Day	Focus	Output
1-2	Python basics	Comfort with syntax, loops, functions, data types, file I/O
3-4	Data cleaning (pandas)	Clean a messy sample CSV end-to-end
5-6	Web scraping tools	Scrape 2-3 pages of books.toscrape.com
7	Applied mini-project	Full scrape + clean + export CSV, shared with Pavanaj

Step 1 — Python Basics (Day 1-2)

Goal: Be able to comfortably write scripts using variables, loops, conditionals, functions, lists/dicts, and file I/O.

Learn here (in order):

- [Official Python Tutorial \(docs.python.org\)](https://docs.python.org) — the authoritative reference; skim sections 1-9, focus on 3-9.
- [freeCodeCamp — Python for Beginners \(full course\)](#) — video + text course, good for hands-on typing.
- [W3Schools Python Tutorial](#) — use as a quick-reference / practice-exercise site alongside the above.

Step-by-step process:

- Day 1 morning: Install Python 3 + VS Code. Learn variables, data types, operators, input/output.
- Day 1 afternoon: Loops (for/while), conditionals (if/elif/else). Do these 10 small exercises:
 - 1. Check if a string contains only digits.
 - 2. Find the median of a list of numbers.
 - 3. Count words longer than N letters in a sentence.
 - 4. Swap two variables without using a temporary variable.
 - 5. Check if a given year is a leap year.
 - 6. Find the common elements between two lists.
 - 7. Convert a list of (key, value) tuples into a dictionary.
 - 8. Compute the running/cumulative sum of a list of numbers.
 - 9. Check if a string has balanced parentheses.
 - 10. Find the most frequently occurring character in a string.
- Day 2 morning: Functions, arguments, return values. Write 5 reusable functions. Lists, dicts, tuples, sets.

- Day 2 afternoon: Reading/writing files (.txt / .csv) with Python's built-in csv module — this bridges straight into Day 3.

Step 2 — Data Cleaning with pandas (Day 3-4)

Goal: Take a messy raw dataset and produce a clean, analysis-ready one.

Learn here:

- [Kaggle — Data Cleaning \(free micro-course\)](#) — 5 short hands-on lessons, best starting point.
- [pandas official Getting Started guide](#) — for syntax reference.
- [Real Python — Data Cleaning With pandas and NumPy](#) — deeper worked example.

Step-by-step process:

- Day 3 morning: Install pandas. Load any sample CSV (e.g. a Kaggle dataset) with `pd.read_csv()`. Handle missing values — `isna()`, `dropna()`, `fillna()`.
- Day 3 afternoon: Fix data types — `astype()`, `pd.to_datetime()`, `pd.to_numeric()`. Remove duplicates, standardize text (`str.strip()`, `str.lower()`).
- Day 4 morning: Detect and handle outliers/inconsistent values; rename & reorder columns.
- Day 4 afternoon: Export the cleaned dataset with `to_csv()` — this is the exact workflow you'll reuse on Day 7.

Extra practice — built-in text datasets (nlTK reuters corpus + sklearn 20-newsgroups):

In addition to a CSV, also practice cleaning raw text using the built-in sample corpora that ship with `nlTK` and `scikit-learn` — no download/scraping needed, good for practicing text-specific cleaning (removing punctuation/stopwords, lowercasing, whitespace normalization, tokenizing).

```
import nltk
nltk.download('reuters')
from nltk.corpus import reuters
fid = reuters.fileids(categories='corn')[0]
raw_text = reuters.raw(fid) # messy raw news-wire text to clean

from sklearn.datasets import fetch_20newsgroups
data = fetch_20newsgroups(subset='train',
categories=['rec.sport.hockey', 'comp.sys.mac.hardware'])
raw_posts = data.data # list of messy raw email/newsgroup posts
```

Task: lowercase the text, strip extra whitespace/newlines, remove punctuation and stopwords, and tokenize into words — then count the most frequent words in each source.

Step 3 — Web Scraping Tools (Day 5-6)

Goal: Fetch a web page with Python and extract structured data from its HTML.

Learn here:

- [Real Python — BeautifulSoup: Build a Web Scraper](#) — best single tutorial to start with (requests + BeautifulSoup).
- [Beautiful Soup official documentation](#) — reference for selectors/methods.
- [requests library Quickstart](#) — how to fetch pages, handle status codes/headers.
- [Selenium with Python — official docs](#) — for pages that need a real browser: JavaScript-rendered content, clicks, scrolling, logins.

- [Playwright for Python — official docs](#) — a faster, more modern alternative to Selenium; same purpose (browser automation), good auto-waiting behaviour.
- [Scrapy official tutorial](#) — optional, only if time allows (Scrapy is the tool of choice for larger, multi-page crawls).

Note on tool choice: books.toscrape.com is plain static HTML, so requests + BeautifulSoup is enough to complete the Day 7 project. Selenium and Playwright are included here because most real-world sites render content with JavaScript and need a real browser to scrape — it's worth doing a short hands-on exercise with each so you're comfortable reaching for them later.

Step-by-step process:

- Day 5 morning: pip install requests beautifulsoup4. Fetch a single page's HTML and print it. Use browser DevTools (Inspect Element) on books.toscrape.com to find the tags/classes holding title, price, rating, stock status.
- Day 5 afternoon: Use BeautifulSoup's find()/find_all() and CSS selectors (select()) to pull those fields for ONE page.
- Day 6 morning: Loop over multiple pages by following the 'next page' URL pattern (the site paginates as /catalogue/page-2.html, etc.). Store data as dicts, write to CSV, add a short delay between requests.
- Day 6 afternoon: pip install selenium playwright (then run `playwright install`). Do one short exercise with each: open books.toscrape.com in an automated browser, click into a book's detail page, and print its title — this is the pattern you'd use on a JS-heavy site where requests/BeautifulSoup alone can't see the content.

Step 4 — Applied Project: Scrape books.toscrape.com (Books priced under £20) — Day 7

Your assigned scope: Scrape every book on the site (across all categories and pages) that is priced under £20 — use the price shown on each book card to filter as you go.

- Go through every category/page on the site and check each book's listed price.
- Collect only books priced under £20 into your dataset.
- Visit each matching book's detail page to capture its full description and UPC/code.

Deliverable: A single clean CSV file with columns: title, price, availability (in stock/out of stock), star rating, category, and product page URL. Share your code + CSV with Pavanaj so your two datasets can be compared and merged into one full dataset.

Practice/reference sandbox: also see [webscraper.io test sites](#) for extra practice once the main task is done.

Notes

- Your exercises and practice datasets are different from Pavanaj's by design, so your work doesn't overlap.
- Only the Day 7 scraping scope differs from your teammate's, so your two outputs can be merged into one full dataset at the end.
- books.toscrape.com is a scraping sandbox with no login/anti-bot restrictions, so no legal or terms-of-service issues arise from practicing on it.
- Do a short daily standup (15 min) with Pavanaj to compare progress and unblock each other on tricky HTML structures.
- Suggested tools/libraries: Python 3.10+, VS Code, pip, requests, beautifulsoup4, pandas, nltk, scikit-learn, selenium, playwright. Optional: Scrapy for anyone who wants to go further.